
CS2306 Lab3

简单的类 MIPS 单周期处理器功能部件的设计与实现（一）

姓名： 吴恒涛 学号： 524031910038

摘要

本报告详述了基于 Vivado 开发平台的类 MIPS 单周期处理器核心控制与运算部件的设计生命周期。实验从指令集架构（ISA）的逻辑抽象入手，利用 Verilog HDL 对主控制单元（Ctr）、ALU 控制单元（ALUCtr）及算术逻辑单元（ALU）进行了纯组合逻辑建模。通过构建完备的 Testbench 进行行为级仿真验证，并深入探讨了硬件描述语言中无关项（Don't Care）的处理策略与状态边界行为。本次实验成功验证了 CPU 译码与运算核心的底层逻辑，为后续构建完整的单周期数据通路奠定了微架构基础。

目录

1 实验导言与微架构概述	2
2 硬件架构与核心逻辑实现	2
2.1 主控制单元模块 (Ctr)	2
2.2 ALU 控制单元模块 (ALUCtr)	3
2.3 核心运算单元 (ALU)	4
3 测试平台构造与行为级仿真自查	4
3.1 主控制器 Ctr 波形自查	4
3.2 ALUCtr 仿真波形与代码差异探讨	5
3.3 ALU 逻辑与 Zero 标志位自查	5
4 工程反思与总结	6

1 实验导言与微架构概述

在单周期 MIPS 处理器的设计范式中，控制信号的生成与数据通路的运算是系统稳定运行的核心。主控制部件负责全局状态的宏观调度，ALU 控制器负责具体算术逻辑操作的微观译码，而 ALU 则作为执行引擎输出最终结果与状态标志。

本次实验聚焦于上述三大核心功能部件的独立设计与仿真。在现阶段，工程实施重点为逻辑层面的正确性验证与波形自查，暂不涉及物理综合、时序约束与底层管脚分配。

2 硬件架构与核心逻辑实现

2.1 主控制单元模块 (Ctr)

主控制单元是指令译码的第一级。其输入直接提取自指令寄存器的最高 6 位 (op-Code)，并输出各类控制信号至数据通路。opCode 与主控单元输出的控制信号功能及真值表映射关系分别如表 1 与表 2 所示。

表 1: 主控单元的控制信号

信号名	功能
regDst	选择目标寄存器, 0 为 rt, 1 为 rd
aluSrc	ALU 的第二个输入, 0 为 rt, 1 为立即数
memToReg	写入数据的来源, 0 为 ALU 结果, 1 为内存
regWrite	寄存器写使能信号
memRead	内存读使能信号
memWrite	内存写使能信号
aluOp	在 ALU 控制单元中进一步处理
branch	当前指令是否为条件分支指令
jump	当前指令是否为跳转指令

表 2: 主控单元的真值表

OpCode	000000	000010	000100	100011	101011
指令	R 型指令	j	beq	lw	sw
aluSrc	0	0	0	1	1
aluOp	10	00	01	00	00
branch	0	0	1	0	0
jump	0	1	0	0	0
memRead	0	0	0	1	0
memToReg	0	0	0	1	0
memWrite	0	0	0	0	1
regDst	1	0	0	0	0
regWrite	1	0	0	1	0

在 Verilog 实现中，采用 always @(*) 敏感列表配合 case 语句进行组合逻辑映射：

```
always @(opCode) begin
    case (opCode)
        6'b000000: begin // R-type 指令：全面使能寄存器与 ALU
            regDst = 1; aluSrc = 0; memToReg = 0;
            regWrite = 1; memRead = 0; memWrite = 0;
            branch = 0; jump = 0; aluOp = 2'b10;
        end
        6'b000100: begin // beq 指令：关闭数据写回，开启分支判定
            regDst = 0; aluSrc = 0; memToReg = 0;
            regWrite = 0; memRead = 0; memWrite = 0;
            branch = 1; jump = 0; aluOp = 2'b01;
        end
    endcase
end
```

```

    end // ... 其他 lw, sw, j 等指令映射略, 详见工程源码
    default: begin // 缺省状态防锁存器生成
        // ... 全部置 0
    end
endcase
end

```

2.2 ALU 控制单元模块 (ALUCtr)

ALU 控制单元从指令中解码出 ALU 需要做什么运算, 一部分工作已经由主控单元完成。指令与控制信号的对应关系如表 3 所示。

表 3: ALU 控制单元信号与指令

指令	aluOp	funct	aluCtr	功能
lw	00	xxxxxx	0010	+
sw	00	xxxxxx	0010	+
beq	01	xxxxxx	0110	-
add	10	100000	0010	+
sub	10	100010	0110	-
and	10	100100	0000	&
or	10	100101	0001	
slt	10	101010	0111	slt

为了使逻辑综合工具能够实现最优的门级化简, 采用了 Verilog 的 `casex` 语法处理带 `x` 的无关项 (Don't Care) 匹配。这种硬件遮罩机制极大精简了代码:

```

always @(aluOp or funct) begin
    casex ({aluOp, funct})
        8'b00xxxxxx: aluCtrOut = 4'b0010; // lw/sw
        8'b01xxxxxx: aluCtrOut = 4'b0110; // beq
        8'b10100000: aluCtrOut = 4'b0010; // R-type: add
        8'b10100010: aluCtrOut = 4'b0110; // R-type: sub
        // ... 其他 R 型译码 (and, or, slt 等) 略
        default: aluCtrOut = 4'b0000;
    endcase
end

```

2.3 核心运算单元 (ALU)

ALU 承担着 CPU 最核心的数据处理任务。除常规算术逻辑外，支持 beq 指令的底层条件跳转判定是其设计的关键。当 ALU 执行减法操作且输出结果全为 0 时，必须异步拉高 Zero 标志位。

```
always @(input1 or input2 or aluCtr) begin
    case (aluCtr)
        4'b0010: aluRes = input1 + input2;
        4'b0110: aluRes = input1 - input2;
        4'b0111: aluRes = (input1 < input2) ? 32'b1 : 32'b0; // slt
        // ... 其他逻辑运算略
        default: aluRes = 32'b0;
    endcase
    // Zero 标志位异步生成逻辑
    if (aluCtr == 4'b0110 && aluRes == 32'b0) zero = 1;
    else zero = 0;
end
```

3 测试平台构造与行为级仿真自查

3.1 主控制器 Ctr 波形自查

在测试驱动中，依次注入了覆盖典型指令的编码序列。从仿真波形可以看出，输出的总线使能均与理论真值表严格对齐，证明主宏观译码逻辑闭环。

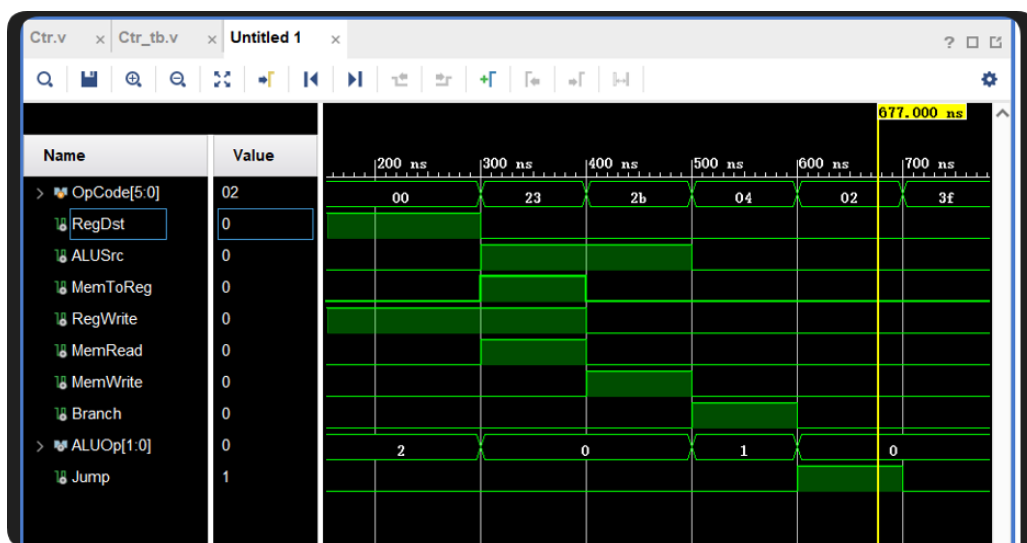


图 1: 主控制单元 Ctr 仿真输出行为波形图

3.2 ALUCtr 仿真波形与代码差异探讨

在 ALUCtr 前仿真中，重点观测了 ALUOp 为 10（R 型指令）时，控制线 aluCtrOut 对 Funct 字段的动态响应，验证了二次译码的准确性。

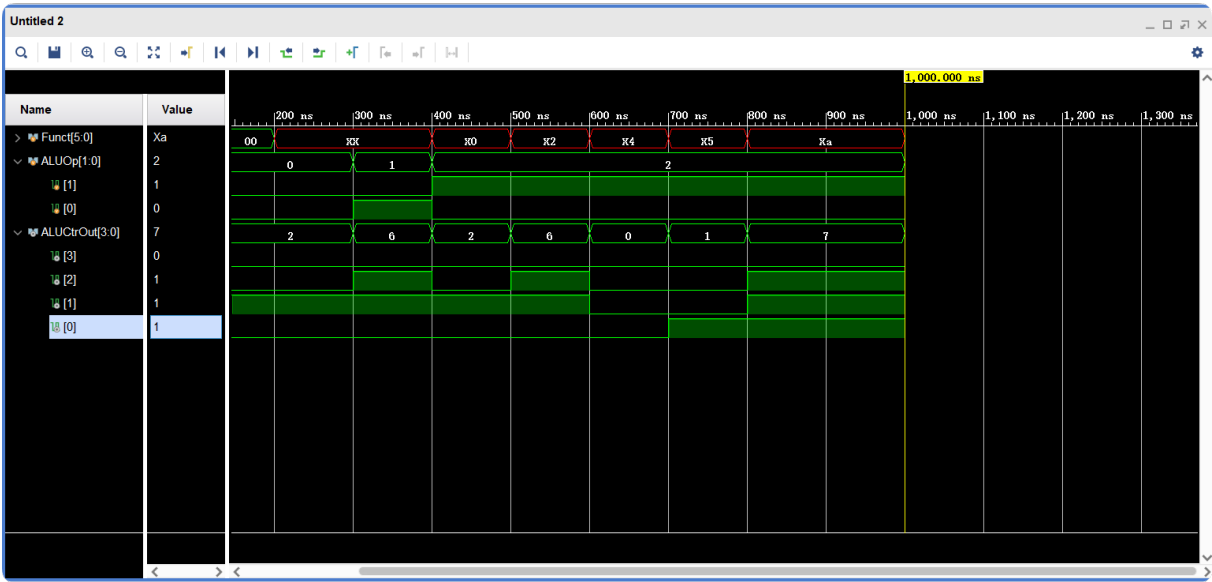


图 2: ALU 控制单元 ALUCtr 仿真行为波形图

【自查点探讨】图 A 与图 B 代码设计差异

结合仿真波形，图 A 与图 B 现象差异的本质在于 Testbench 对硬件无关项（Don't Care）的赋值策略不同：

- **图 A 策略：**主动向 Funct 端口注入不定态 XXXXXX（表现为红色波形）。但得益于代码中 casex 的硬件遮罩效应，译码器有效屏蔽了这些不定态，依然输出了正确的使能值，体现了极佳的逻辑鲁棒性。
- **图 B 策略：**在任何周期都向 Funct 端口输入了确切的二进制码。虽然此时低位字段被译码器逻辑无视，但因为有明确的驱动信号，波形线上消除了红色不定态标记，更加符合实际物理总线的连线状态。

3.3 ALU 逻辑与 Zero 标志位自查

为了验证条件跳转的核心依据，特意在仿真中构造了减法运算（aluCtr = 0110）且两操作数相等的临界条件。

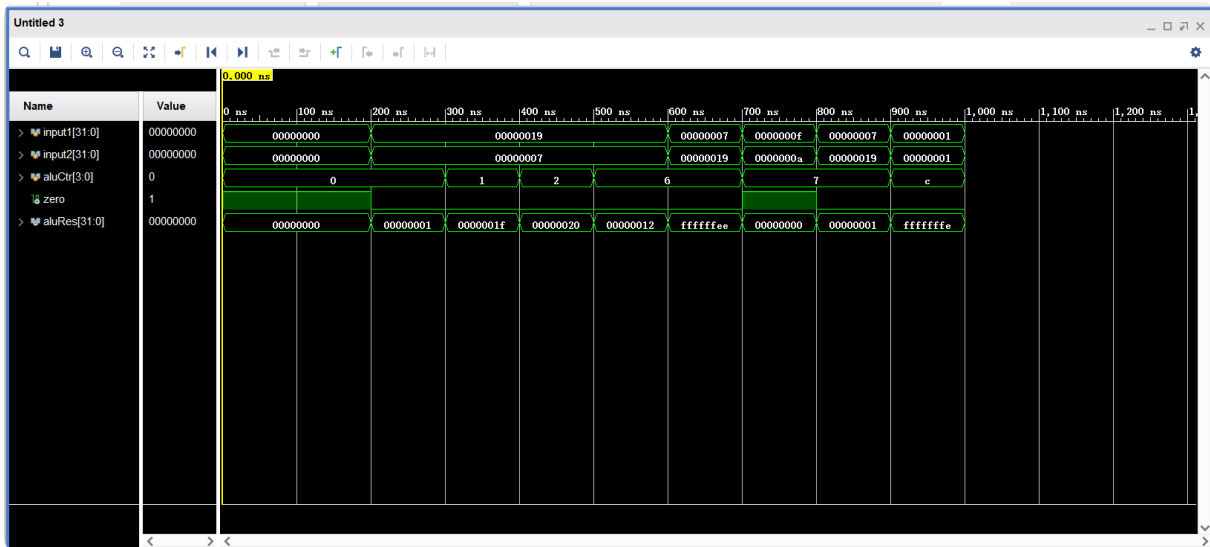


图 3: ALU 模块组合逻辑行为仿真波形图（含 Zero 触发验证）

从波形图可以直观地确认，当运算结果 `aluRes` 下降为 0 的同一时间沿，Zero 输出引脚迅速拉高。该硬件动作完美契合了 `beq` 指令所需的底层判定逻辑。

4 工程反思与总结

经过 Lab1、Lab2 的训练，我已经能较为熟练地进行硬件建模。本次实验完成了从指令集架构向底层 RTL 代码映射的过程。在纯组合逻辑的构建中，我最深刻的体会是“硬件视角的并发性”与“无关项 (X) 的工程价值”。合理运用 `casex` 语法处理 Don't Care 项，不仅能精简代码量，更能有效指导综合工具优化逻辑门资源。此外，基于“局部功能自治”的模块化自查策略，帮助我理清了数据与控制流的分层边界。